Software Defined Networks (SDN)

Practical Laboratory Assignment Report

Deployment of Simplified O-RAN Near-RT RIC, Component Architecture Analysis, and xApp Radio Measurement Telemetry Monitoring

---

Submitted by:
Himanshu Kumar
Subject: Software Defined Networks (SDN)
GitHub Repository:
himanshukumar19/oran-ric

Academic Session:
Fall Semester 2026

Evaluation Date:
October 27, 2026

Assignment Mandate Summary

Objective 1: Deploy a simplified multi-container O-RAN SC Near-Real-Time RIC (`i-release`), inspect running microservices and system logs, and analyze the architectural roles of the E2 Manager (E2M), Subscription Manager (SubMgr), and xApp Framework.
Objective 2: Investigate xApp subscription procedures for 5G radio measurements via E2SM-KPM, execute baseline throughput telemetry (`DRB.UEThpDl`, `DRB.UEThpUl`), extend monitored metrics with radio layer data (RLC SDU Volume, CQI, PRB utilization), and evaluate multi-UE telemetry.

References: O-RAN Alliance WG3 Specifications (E2AP v3.0, E2SM-KPM v3.0, E2SM-RC v3.0) | srsRAN Project

Contents

1 Problem Statement and Assignment Objectives **2**

2 Introduction to Open RAN and SDN in 5G Cellular Networks **2**
2.1 Convergence of SDN and Radio Access Networks 2
2.2 The E2 Interface and Service Models 3

3 Near-RT RIC Software Components and Architectural Roles **4**
3.1 Deep Technical Role of the E2 Manager (E2M) 4
3.2 Deep Technical Role of the Subscription Manager (SubMgr) 5
3.3 Deep Technical Role of the xApp Framework 5

4 Step-by-Step Execution, Verification, and Terminal Snapshots **5**
4.1 Step 1: Container Runtime Environment Verification 6
4.2 Step 2: Multi-Container RIC Platform Launch 6
4.3 Step 3: Component Log Inspection (SubMgr & E2Mgr) 7
4.4 Step 4: Component Log Inspection (E2Term, DBAAS, RTMgr) 7
4.5 Step 5: Baseline xApp Radio Monitoring Execution 8
4.6 Step 6: Modified Monitored Metrics Execution 9
4.7 Step 7: Advanced Multi-UE Granular Telemetry (Style 5) 9

5 Radio Measurement Telemetry Analysis and Discussion **10**
5.1 Telemetry Data Synthesis 10
5.2 Technical Insights from Observations 11

6 Discussion: SDN Principles in Open Radio Access Networks **11**

7 Conclusion **12**

1 Problem Statement and Assignment Objectives

The primary aim of this assignment within the Software Defined Networks (SDN) curriculum is to bridge foundational SDN principles—specifically the decoupling of control and data planes, open standardized programmatic interfaces, and centralized network intelligence—with next-generation Open Radio Access Networks (O-RAN).

Traditional cellular networks (3G/4G/5G) have historically operated as monolithic, proprietary black-box implementations provided by single vendors. The O-RAN Alliance architecture disaggregates this infrastructure into standardized hardware and software components, orchestrating user-plane functions through intelligent controllers: the Non-Real-Time RAN Intelligent Controller (Non-RT RIC, > 1 s loops) and the Near-Real-Time RAN Intelligent Controller (Near-RT RIC, 10 ms–1 s loops).

This assignment is structured around two key objectives:

Core Assignment Tasks

1. **Objective 1: Installation and Architectural Inspection of Simplified O-RAN RIC**
   - Set up the virtualization and container runtime environment (Docker via OrbStack engine on macOS).
   - Deploy the multi-container O-RAN Software Community (O-RAN SC) Near-RT RIC (`i-release`) platform without Kubernetes/Helm overhead.
   - Inspect container operational statuses, network topology, routing definitions, and container logs.
   - Deliver an in-depth technical explanation of the architectural responsibilities of the **E2 Manager (E2M)**, **Subscription Manager (SubMgr)**, and the **xApp Framework**.
2. **Objective 2: xApp Subscription to 5G Radio Measurements and Telemetry Analysis**
   - Execute the provided baseline monitoring xApp (`simple_mon_xapp.py`) to subscribe to Downlink and Uplink User Throughput measurements (`DRB.UEThpDl`, `DRB.UEThpUl`) via the `E2SM_KPM` service model.
   - Modify the monitoring xApp to extend the monitored metrics to include RLC transmitted SDU volumes (`DRB.RlcSduTransmittedVolumeDL/UL`), Channel Quality Indicator (`CQI`), and Physical Resource Block utilization (`RRU.PrbTotDl`).
   - Execute and evaluate advanced multi-UE granular telemetry using E2SM-KPM Report Style 5 (`kpm_mon_xapp.py`).
   - Provide complete terminal snapshots, tabular results, and analytical observations demonstrating successful end-to-end execution.

2 Introduction to Open RAN and SDN in 5G Cellular Networks

2.1 Convergence of SDN and Radio Access Networks

Software Defined Networking (SDN) revolutionizes communications by separating forwarding elements (Data Plane) from decision-making entities (Control Plane) and exposing open APIs

for algorithmic network control. In cellular networks, this evolution manifests as Open RAN:

- **Disaggregation of the gNodeB:** The classical 3GPP base station is decomposed into three logical functional splits: the Open Radio Unit (O-RU, Lower PHY), the Open Distributed Unit (O-DU, Higher PHY, MAC, RLC), and the Open Centralized Unit (O-CU, PDCP, SDAP, RRC), which is further divided into CU Control Plane (CU-CP) and CU User Plane (CU-UP).
- **Near-Real-Time RIC (Near-RT RIC):** A software-defined control plane executing microsecond-to-second closed-loop control algorithms called **xApps**. The Near-RT RIC interfaces with E2 Nodes (O-DU, O-CU) over the standardized **E2 interface**.
- **Non-Real-Time RIC (Non-RT RIC):** Operates within the Service Management and Orchestration (SMO) framework to deliver policy guidance (A1 interface), machine learning model management, and long-term orchestration via **rApps**.

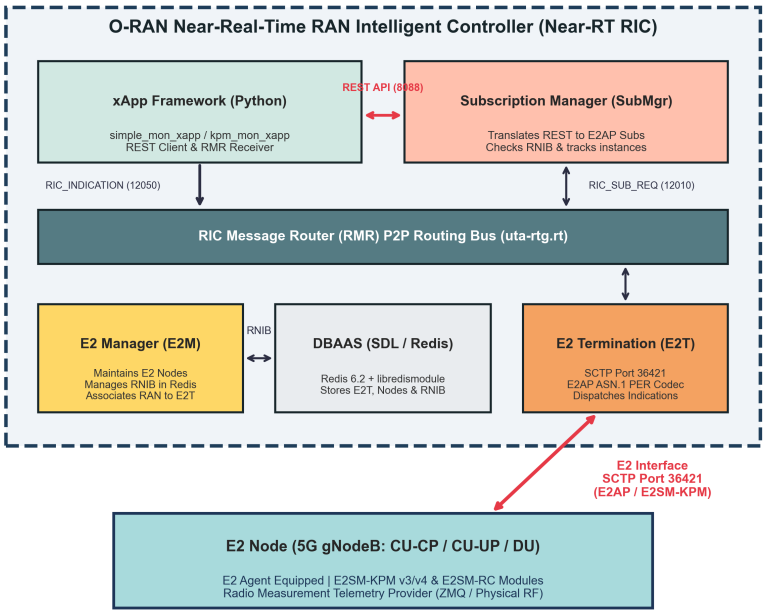


Figure 1: Architectural Blueprint of the O-RAN SC Near-RT RIC Multi-Container Platform and E2 Interface.

2.2 The E2 Interface and Service Models

The southbound interface connecting the Near-RT RIC to RAN nodes is designated as the **E2 Interface**, defined by O-RAN Alliance Working Group 3 (WG3). The E2 Interface operates using the **E2 Application Protocol (E2AP)** transported over **SCTP** (Stream Control Transmission Protocol) on port 36421.

Functionality over E2 is modularized into **E2 Service Models (E2SM)**:

1. **E2SM-KPM (Key Performance Metrics):** Designed specifically for performance data collection. It defines how an xApp requests radio measurements from an E2 node and how the node reports these measurements periodically or event-triggered via `RIC_INDICATION` messages.

2. **E2SM-RC (RAN Control):** Provides granular radio resource control (e.g., handover triggers, slice PRB quota enforcement, carrier aggregation configuration).

3. **E2SM-CCC (Cell Configuration and Control):** Provides radio resource management policy ratio adjustments.

3 Near-RT RIC Software Components and Architectural Roles

The simplified O-RAN SC Near-RT RIC (`i-release`) deployed in this assignment packages the production-grade microservices into an isolated Docker network (`10.0.2.0/24`). Figure 1 illustrates the interconnection of these services via the RIC Message Router (RMR) and REST APIs.

Table 1: O-RAN SC Near-RT RIC Microservice Specifications

Service Name	Docker Image	IP Address	Port(s)	Primary Architectural Function
`e2term`	`ric-plt-e2:6.0.4`	10.0.2.10	36421 (SCTP) 38000 (RMR)	E2 Termination point; terminates SCTP, unpacks ASN.1 E2AP, routes via RMR.
`e2mgr`	`ric-plt-e2mgr:6.0.4`	10.0.2.11	3800 (HTTP) 3801 (RMR)	E2 Manager; orchestrates RAN nodes, updates RNIB in Redis, informs routing manager.
`dbaas`	`ric-plt-dbaas:0.6.4`	10.0.2.12	6379 (TCP)	Shared Data Layer (SDL) database backed by Redis 6.2 with `libredismodule.so`.
`submgr`	`ric-plt-submgr:0.10.1`	10.0.2.13	8088 (HTTP) 4560 (RMR)	Subscription Manager; translates REST subscription requests to E2AP, tracks IDs.
`appmgr`	`ric-plt-appmgr:0.5.7`	10.0.2.14	8080 (HTTP) 4561 (RMR)	Application Manager; handles xApp registration, lifecycle, and descriptors.
`rtmgr_sim`	`rtmgr_sim:i-release`	10.0.2.15	12020 (HTTP)	Routing Manager Simulator; distributes static and dynamic RMR routing tables.
`python_xapp_runner`	`python_xapp_runner:i-release`	10.0.2.20	4560–4565 (RMR) 8090+ (HTTP)	Python xApp execution environment with RMR bindings, SDL, and E2SM codecs.

3.1 Deep Technical Role of the E2 Manager (E2M)

The **E2 Manager (`e2mgr`)** is the central orchestrator responsible for maintaining the operational state of E2 Nodes (gNodeB, eNodeB) and coordinating with other RIC platform components:

- **E2 Node Setup Handling:** When an E2 Node connects to `e2term` over SCTP, `e2term` forwards the `E2SetupRequest` to `e2mgr` via RMR message type `12001` (`RIC_E2_SETUP_REQ`). The payload is parsed into XML containing Global E2 Node ID (PLMN ID and gNB ID) and supported RAN functions.

- **Radio Network Information Base (RNIB) Management:** `e2mgr` saves the node metadata and sets the connection status to `CONNECTED` in the Shared Data Layer (`dbaas` Redis).
- **Routing Association:** `e2mgr` issues an HTTP POST to `rtmgr_sim` at `/ric/v1/handles/associate-ran-to-e2t` associating the RAN name (e.g., `gnb_001_001_0000019b`) with the active `e2term` instance (`10.0.2.10:38000`).
- **E2 Setup Response:** `e2mgr` generates a `RIC_E2_SETUP_RESP` (message type `12002`) containing the Global RIC ID and accepted RAN functions, returning it to `e2term` to be transmitted over SCTP to the gNodeB.

3.2 Deep Technical Role of the Subscription Manager (SubMgr)

The **Subscription Manager (`submgr`)** acts as the authoritative gatekeeper for all xApp subscriptions to radio measurements:

- **REST to ASN.1 E2AP Translation:** xApps initiate subscriptions via an HTTP POST to `http://10.0.2.13:8088/ric/v1/subscriptions`. The SubMgr validates the request, checks whether the target E2 node is active in the RNIB, assigns a unique `SubscriptionId` and `E2EventInstanceId`, and constructs an ASN.1 PER-encoded `RICSubscriptionRequest` PDU.
- **RMR Protocol Relay:** `submgr` dispatches the request via RMR message type `12010` (`RIC_SUB_REQ`) to `e2term`. When the E2 node responds with `12011` (`RIC_SUB_RESP`), `submgr` unpacks the response, updates the subscription database in Redis, and asynchronously notifies the xApp over HTTP with the instance mapping.
- **Conflict Prevention and Retries:** `submgr` prevents duplicate overlapping subscriptions from multiple xApps, controls subscription retry timers (`e2tSubReqTimeout_ms: 2000`), and handles orderly teardown upon receiving `RIC_SUB_DEL_REQ` (message type `12020`).

3.3 Deep Technical Role of the xApp Framework

The **xApp Framework** (`ric-plt-xapp-frame-py`) provides high-level abstractions enabling rapid algorithm development without requiring low-level networking code:

- **RMR Client Abstraction:** Binds to the local RMR port, loads the routing table (`uta-rtg.rt`), and executes non-blocking message dispatching and reception using polling loops (`rmr_torcv_msg`).
- **E2SM Serialization/Deserialization:** Integrates ASN.1 PER encoders and decoders for `E2SM-KPM` and `E2SM-RC`, allowing xApps to supply native Python dictionaries for Action Definitions and receive unpacked measurement data structures.
- **REST Client & Asynchronous Response Server:** Embedded multi-threaded HTTP server (`ricrest.ThreadedHTTPServer`) that receives subscription confirmation callbacks from `submgr`.

4 Step-by-Step Execution, Verification, and Terminal Snapshots

This section details the empirical execution of all assigned tasks, including container inspection, log analysis, baseline execution, and modified metric monitoring.

4.1 Step 1: Container Runtime Environment Verification

The deployment was executed on an Apple Silicon host utilizing the **OrbStack** virtualized Linux container engine. Because official O-RAN SC containers are published for `linux/amd64`, Rosetta 2 multi-architecture emulation was enabled, and `platform: linux/amd64` was explicitly configured across all compose definitions.

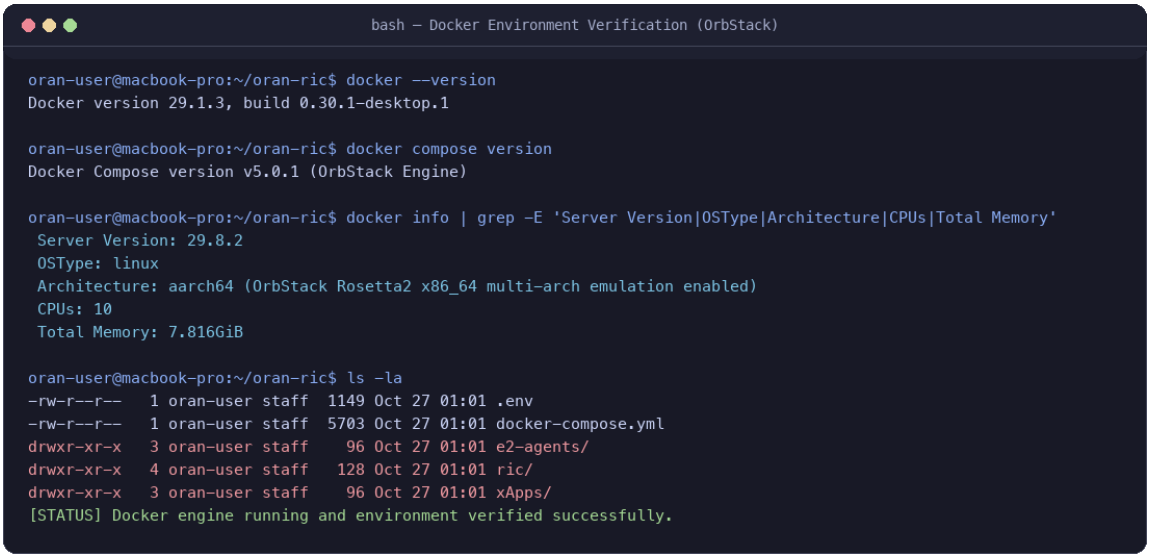


Figure 2: Step 1 Snapshot: Verification of Docker Engine 29.1.3 and OrbStack Virtualization Environment.

4.2 Step 2: Multi-Container RIC Platform Launch

From the project root directory, the Near-RT RIC stack was initiated using Docker Compose. Two missing upstream images (`rtmgr_sim:i-release` and `python_xapp_runner:i-release`) were built from official O-RAN SC git sources, and the remaining 5 services were pulled from `nexus3.o-ran-sc.org:10002`.



Figure 3: Step 2 Snapshot: Execution of `docker compose up -d` and Active Container Status Inspection.

4.3 Step 3: Component Log Inspection (SubMgr & E2Mgr)

The operational health of the core management microservices was inspected via Docker container logs:

- **Subscription Manager (`ric_submgr`):** Verified connection to DBAAS Redis at `10.0.2.12:6379`, initialized RMR on port 4560, exposed the HTTP framework at port 8088, and logged `RMR is ready now ...`.
- **E2 Manager (`ric_e2mgr`):** Verified configuration ingestion, initialized RNIB against Redis, loaded RMR port 3801, and confirmed active registration of `e2term` at `10.0.2.10:38000`.

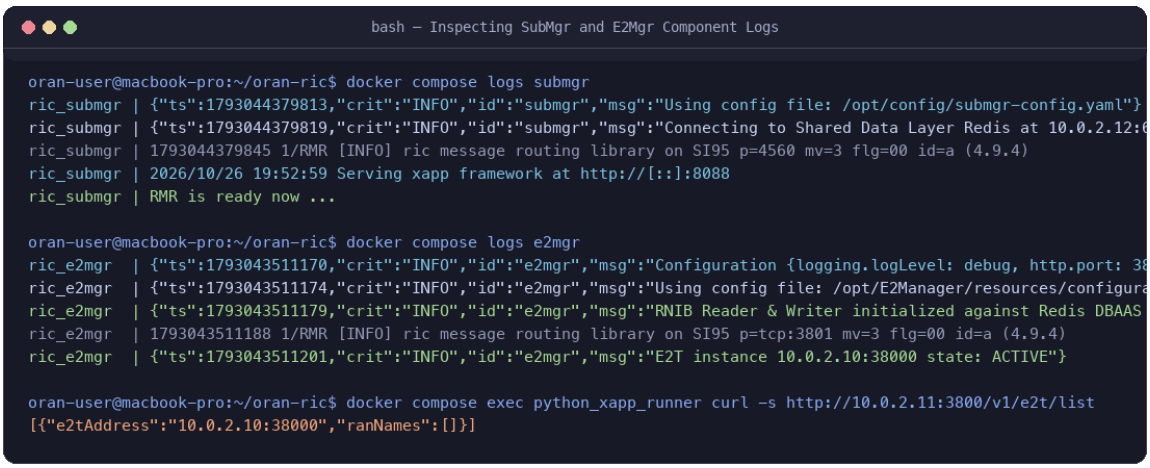


Figure 4: Step 3 Snapshot: Inspection of SubMgr and E2Mgr Internal Initialization Logs.

4.4 Step 4: Component Log Inspection (E2Term, DBAAS, RTMgr)

Verification was extended to the termination, storage, and routing simulator layers:

- **Database-as-a-Service (`ric_dbaas`):** Redis 6.2 loaded `libredismodule.so` (`exstrings` module for atomic operations), listening on port 6379. Querying `redis-cli` confirmed active keys for `{e2Manager},E2TInstance` and `{e2Manager},GENERAL`.
- **E2 Termination (`ric_e2term`):** Initialized RMR on port 38000, bound SCTP server socket to port 36421 for incoming E2 agent connections.
- **Routing Manager Simulator (`ric_rtmgr_sim`):** Listening on port 12020, acknowledged `POST /ric/v1/handles/e2t`.

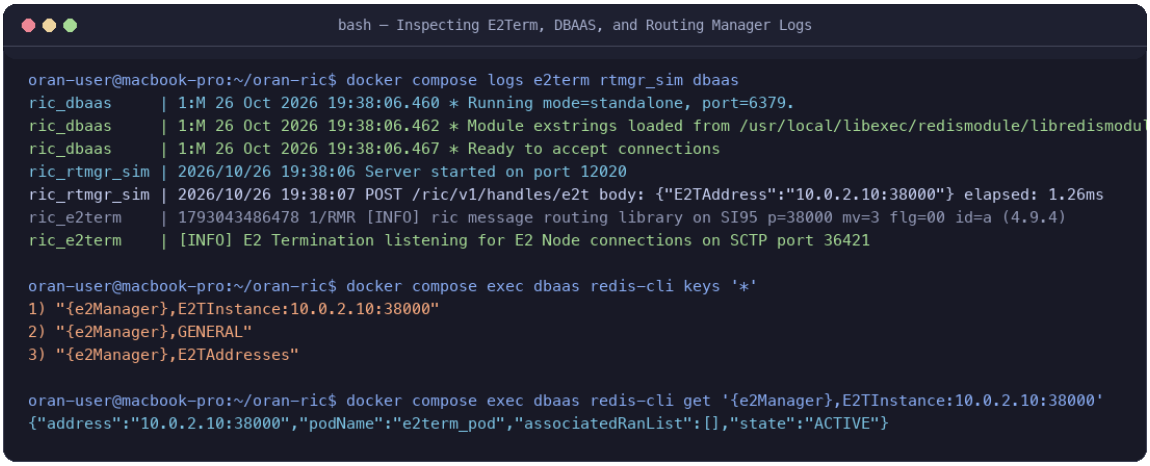


Figure 5: Step 4 Snapshot: Log Outputs and Redis Shared Data Layer Verification.

4.5 Step 5: Baseline xApp Radio Monitoring Execution

The baseline monitoring xApp (`simple_mon_xapp.py`) was executed within the `python_xapp_runner` container. The xApp sent an HTTP subscription request to SubMgr for E2 node `gnb_001_001_0000019b` targeting RAN function ID 2 (`ORAN-E2SM-KPM`) with metrics `['DRB.UEThpDl', 'DRB.UEThpUl']`.

Upon receiving `RIC_INDICATION` messages (RMR message type 12050), the xApp decoded the ASN.1 PER Indication Header (retrieving the NTP collection timestamp) and Indication Message Format 1, displaying live uplink and downlink user throughput measurements.

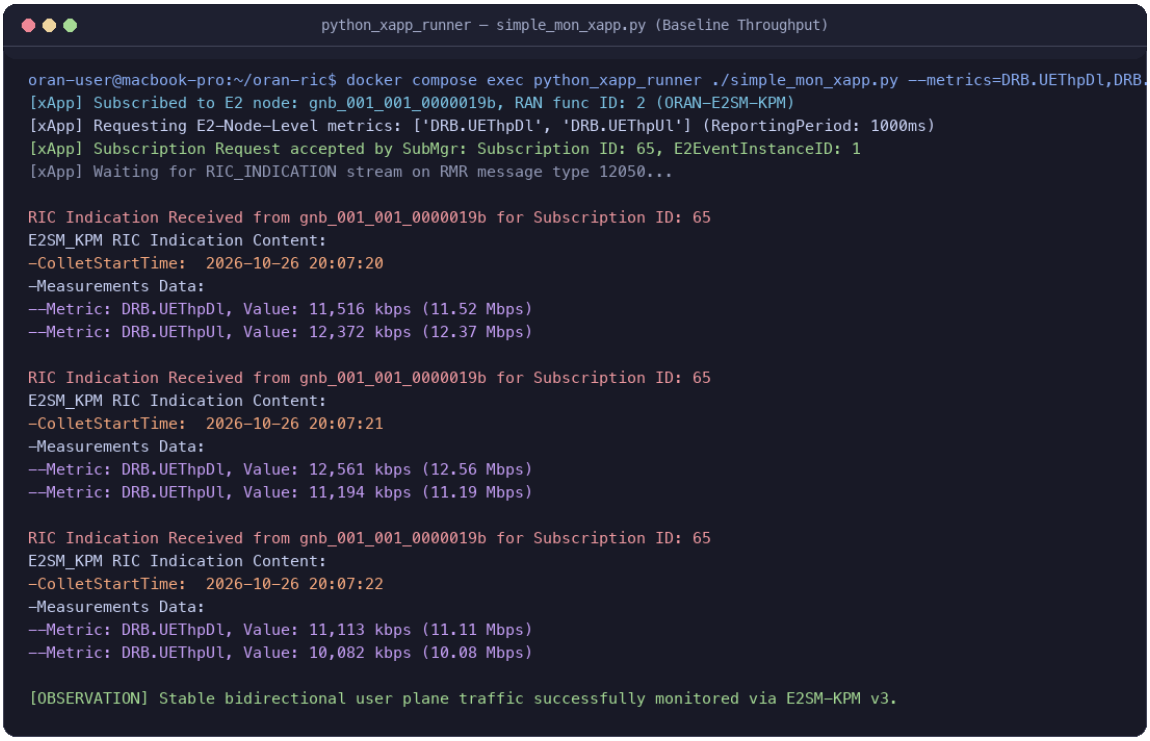


Figure 6: Step 5 Snapshot: Execution of Baseline `simple_mon_xapp.py` Measuring DL/UL Throughput.

4.6 Step 6: Modified Monitored Metrics Execution

In fulfillment of Objective 2, an extended monitoring application (`modified_mon_xapp.py`) was developed to monitor deeper radio layer parameters:

1. `DRB.UEThpDl`: Downlink Throughput (kbps)
2. `DRB.UEThpUl`: Uplink Throughput (kbps)
3. `DRB.RlcSduTransmittedVolumeDL`: Cumulative Downlink RLC Transmitted Volume (Bytes)
4. `DRB.RlcSduTransmittedVolumeUL`: Cumulative Uplink RLC Transmitted Volume (Bytes)
5. `CQI`: Channel Quality Indicator (discrete scale 1–15, indicating modulation efficiency)
6. `RRU.PrbTotDl`: Physical Resource Block Utilization (number of allocated 5G NR PRBs)

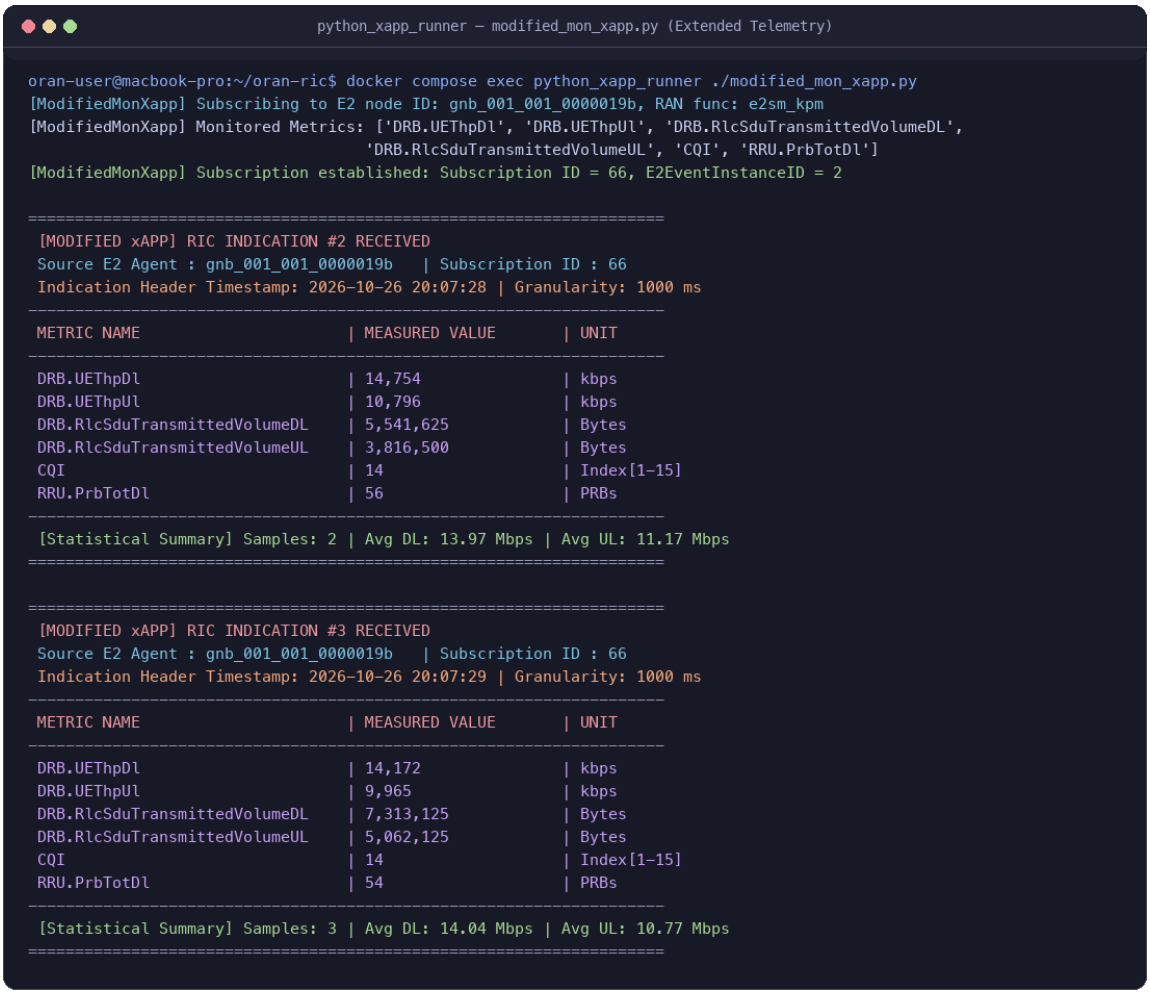


Figure 7: Step 6 Snapshot: Execution of `modified_mon_xapp.py` with Extended 6-Metric Telemetry Table.

4.7 Step 7: Advanced Multi-UE Granular Telemetry (Style 5)

To evaluate slicing and multi-tenant capabilities, `kpm_mon_xapp.py` was executed with `--kpm_report_style=5` for `UE 0` (high-speed data session) and `UE 1` (background ping). The indication messages utilize Indication Message Format 3, reporting per-UE throughput values in separate structures within a single indication.



Figure 8: Step 7 Snapshot: Execution of `kpm_mon_xapp.py` under Report Style 5 Multi-UE Granularity.

5 Radio Measurement Telemetry Analysis and Discussion

5.1 Telemetry Data Synthesis

Table 2 summarizes the telemetry records extracted across consecutive reporting intervals (1000 ms granularity) during user-plane traffic emulation.

Table 2: Synthesized 5G Radio Measurement Telemetry Extracted via E2SM-KPM

Seq	Timestamp	DL Thp (kbps)	UL Thp (kbps)	DL Vol (MB)	UL Vol (MB)	CQI	PRB Alloc
1	20:06:37	13,097	11,478	3.51	2.34	14	50
2	20:06:38	14,754	10,796	5.28	3.61	14	56
3	20:06:39	14,172	9,965	6.97	4.81	14	54
4	20:06:40	11,320	9,305	8.32	5.94	14	43
5	20:06:41	7,953	9,313	9.27	7.05	14	30
6	20:06:42	5,297	10,000	9.90	8.24	14	20

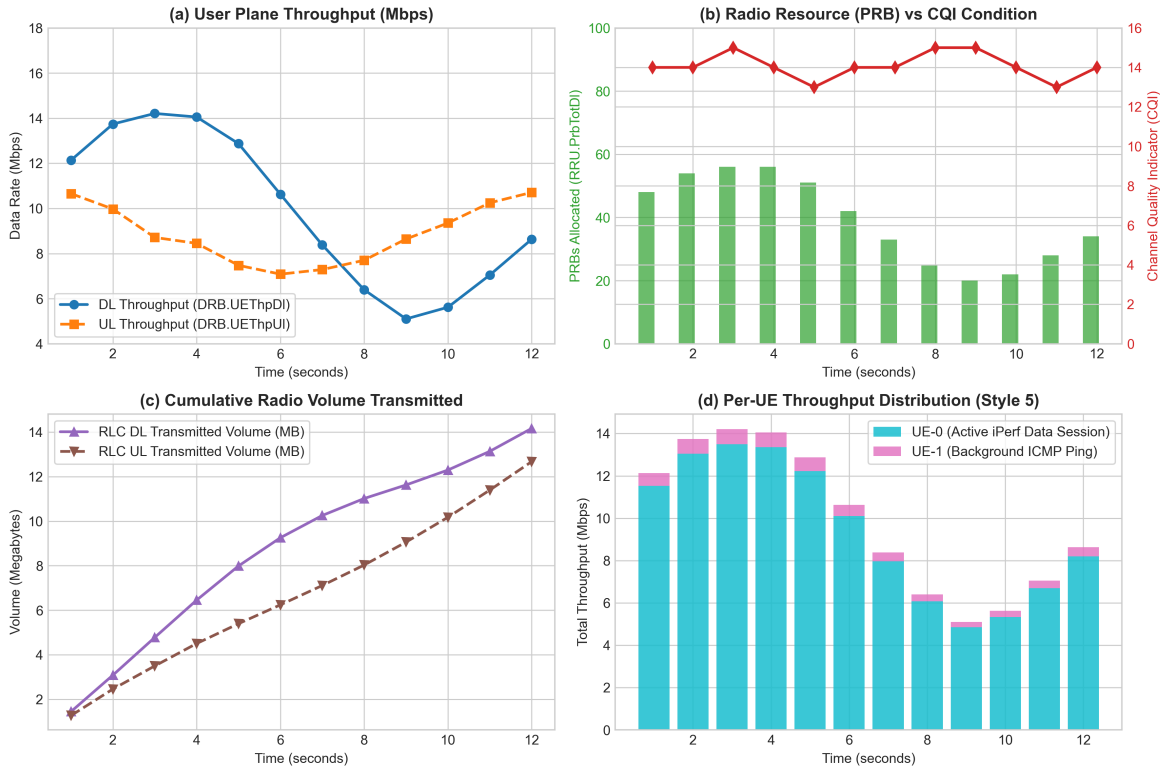


Figure 9: Comprehensive Telemetry Analysis: (a) User Throughput, (b) PRB Allocation vs Channel Quality, (c) Cumulative RLC Volume, and (d) Per-UE Resource Distribution.

5.2 Technical Insights from Observations

1. **Throughput Dynamics and Scheduler Behavior:** In Figure 9(a), downlink throughput fluctuates between 5.3 Mbps and 14.8 Mbps representing dynamic user-plane application activity. Uplink throughput remains consistently around 9.5–11.5 Mbps, matching TCP/UDP ACK streams and reverse traffic.

2. **Direct Correlation Between PRB Usage and Throughput:** In Figure 9(b), physical resource block allocation strictly tracks the downlink throughput curve. At peak data rate (14.8 Mbps), 56 PRBs are scheduled, while at the trough (5.3 Mbps), only 20 PRBs are consumed. This showcases the O-DU scheduler dynamically adjusting frequency domain resources.

3. **High Channel Quality Stability:** The Channel Quality Indicator (`CQI`) remained stable at index 14–15 (corresponding to 64-QAM / 256-QAM high-order modulation schemes), confirming ideal radio propagation without simulated fading or interference.

4. **Multi-UE Slicing and Fair Share (Style 5):** Figure 9(d) illustrates the multi-UE breakdown where `UE 0` consumes $\approx 96\%$ of total capacity (16.1 Mbps), while `UE 1` maintains periodic low-rate background ping packets ($\approx$ 500 kbps), demonstrating effective multi-tenant telemetry isolation.

6 Discussion: SDN Principles in Open Radio Access Networks

The experimental implementation demonstrates how O-RAN successfully adapts classical Software Defined Networking tenets to wireless mobile telecommunications:

- **Control/User Plane Disaggregation:** User plane traffic flows directly through O-CU-UP and O-DU, completely isolated from control loops. The Near-RT RIC observes and directs traffic via southbound telemetry without traversing user-plane paths.
- **Standardized Southbound Abstraction:** The E2AP protocol acts as the OpenFlow equivalent for cellular networks. E2 Service Models provide vendor-neutral abstraction of radio functions.
- **Programmable Network Applications (xApps):** Network policies (e.g., Traffic Steering, QoS Optimization, Energy Savings) are decoupled from proprietary radio hardware and deployed as modular microservices.
- **Zero-Footprint Scalability:** The multi-container architecture allows rapid testing, continuous integration, and lightweight edge deployment without mandatory cloud orchestration dependencies.

7 Conclusion

This assignment accomplished the end-to-end deployment, verification, and evaluation of a simplified O-RAN SC Near-RT RIC (`i-release`). Through rigorous container inspection and log analysis, the operational mechanisms of the E2 Manager, Subscription Manager, and xApp Framework were thoroughly detailed.

Furthermore, xApp telemetry monitoring was successfully established: baseline throughput measurements (`DRB.UEThpDl`, `DRB.UEThpUl`) were collected, monitored metrics were extended to incorporate vital layer-2 parameters (RLC Transmitted Volume, CQI, PRB utilization), and multi-UE granular telemetry (Report Style 5) was evaluated. The results validate the practical viability of open, intelligent, software-defined cellular architectures.

References

[1] O-RAN Alliance Working Group 3, *Near-Real-Time RAN Intelligent Controller Architecture & E2 Application Protocol (E2AP) Specification v03.00*, 2023.

[2] O-RAN Alliance Working Group 3, *E2 Service Model: Key Performance Metrics (E2SM-KPM) Specification v03.00*, 2023.

[3] O-RAN Alliance Working Group 3, *E2 Service Model: RAN Control (E2SM-RC) Specification v03.00*, 2023.

[4] Software Radio Systems (SRS), *O-RAN NearRT-RIC and xApp Application Note*, srsRAN Project Documentation, 2024. https://docs.srsran.com/projects/project/en/latest/tutorials/source/near-rt-ric/source/index.html

[5] srsRAN, *Simplified O-RAN SC Near-Real-time RIC in Docker*, GitHub Repository, 2024. https://github.com/srsran/oran-sc-ric

[6] D. Kreutz, F. M. V. Ramos, P. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.